Bilkent University
Computer Engineering
Senior Design Project

IssuePointer

Project Specification Document

Alphan Eker
Şeymanur Kılıç
Serdar Özata
Berkan Şahin
Efe Yakar
Supervisor: Eray Tüzün

December 15, 2023

Contents

1 Introduction **2**
1.1 Description . . . 2
1.2 High-Level System Architecture of the Proposed Solution . . . 2
1.2.1 Frontend Architecture . . . 2
1.2.2 Backend Architecture . . . 4
1.3 Constraints . . . 4
1.3.1 Implementation Constraints . . . 4
1.3.2 Economic Constraints . . . 5
1.4 Professional and Ethical Issues . . . 5

2 Design Requirements **5**
2.1 Functional Requirements . . . 5
2.1.1 Duplicate Reports . . . 5
2.1.2 Bug Localization . . . 5
2.1.3 Automated Program Repair . . . 6
2.2 Non-functional Requirements . . . 6
2.2.1 Accessibility . . . 6
2.2.2 Modularity . . . 6
2.2.3 Privacy . . . 6
2.2.4 Security . . . 6
2.2.5 Performance . . . 6

3 Feasibility Discussions **6**
3.1 Market & Competitive Analysis . . . 6
3.2 Academic Analysis . . . 7

1 Introduction

1.1 Description

Resolving bug reports is a time-consuming process for developers, and duplicate bug reports lead to wasted time in large projects, especially for developers new to the project. The scope of the project is to aid developers in resolving bug reports by finding similar issues and pointing the developers toward possible solutions. This project scans the source code and extracts data from previous bug reports to find similar issues in the past, if they exist, find code segments that are suspected of introducing the problem in a given bug report, and possibly suggest solutions based on past issues and their solutions.

The project will be modular, with multiple frontend modules for version control platforms and integrated development environments (Visual Studio Code, GitHub, etc.).

Bug reports are not immediately processed by developers. Hence, we do not have strict time constraints. However, we still need to be computationally efficient because of cost constraints.

1.2 High-Level System Architecture of the Proposed Solution

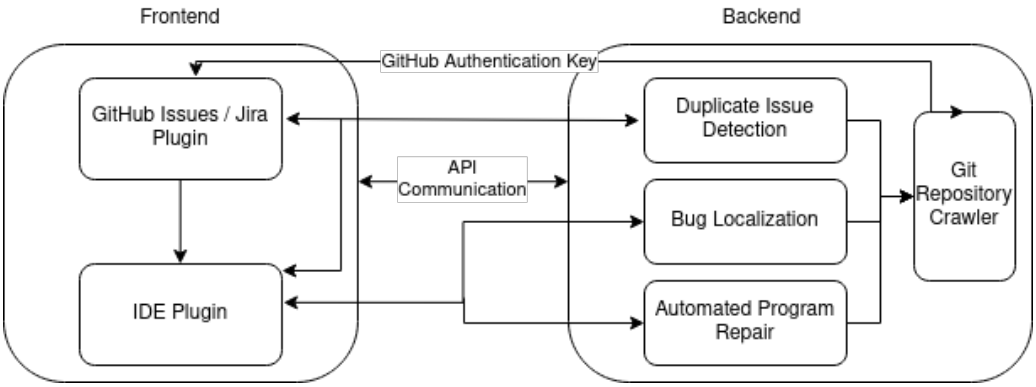

Figure 1: Diagram of IssuePointer components

1.2.1 Frontend Architecture

The frontend client uses an authentication key to send requests to the IssuePointer backend. Users will use the same authentication key across different plugins throughout their workflow.

Github Issues/Jira Plugin The Github Issues and Jira plugins will comment on new issues, showing possible duplicate past bug reports along with confidence values obtained from the backend. Users will be able to click the issue names in the comment to be directed to their IDE. Following their solution, the client will be able to give feedback if the solution was helpful for the solution.

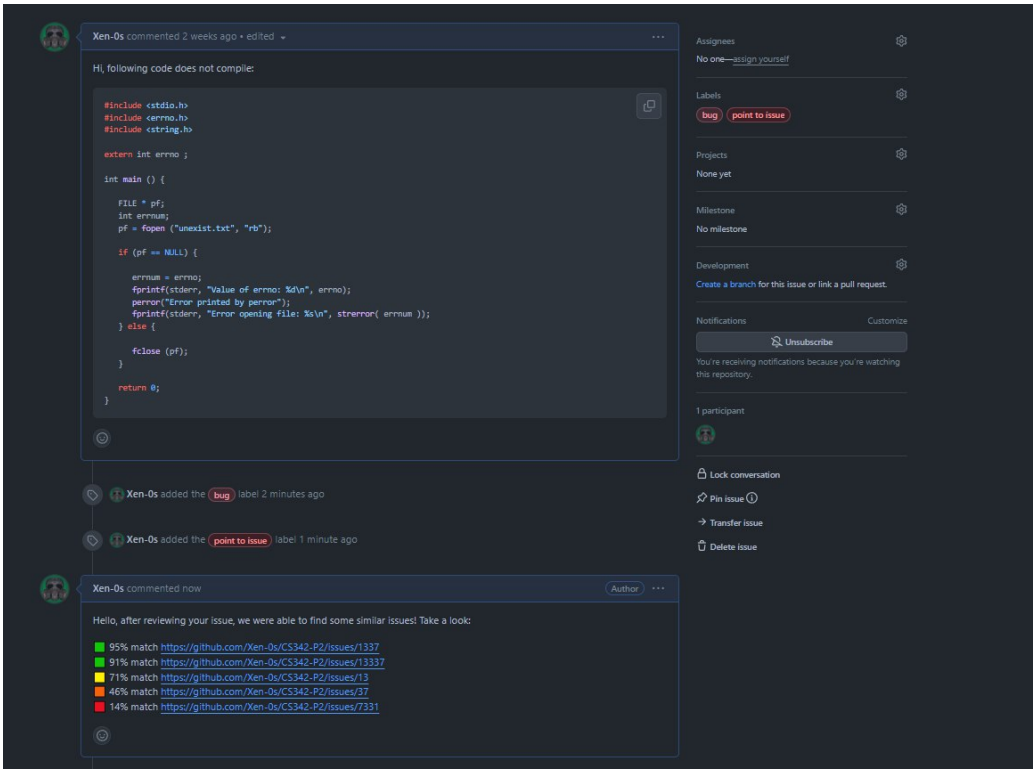

Figure 2: Mock-up of IssuePointer GitHub user interface

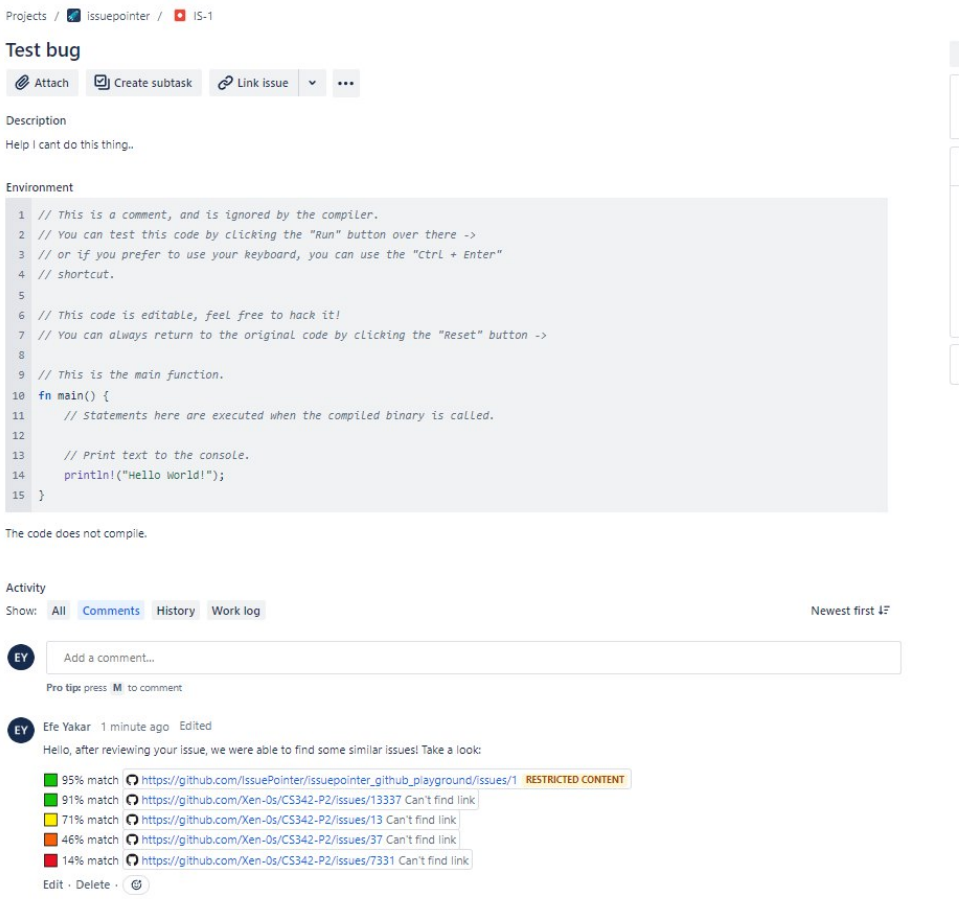

Figure 3: Mock-up of IssuePointer Jira user interface

IDE Plugin The client's software keeps track of the issue the user has chosen. While using Bug Localization or Automated Program Repair, the client sends requests along with the kept issue to obtain information fluidly without asking the client about the issue that they are trying to solve.

- **Bug Localization Visualization:** The suggested lines are shown along with the confidence level and are navigable via shortcuts and next/previous icons. Lines are indicated with purple color, similar to IDEs indicating new lines with green.

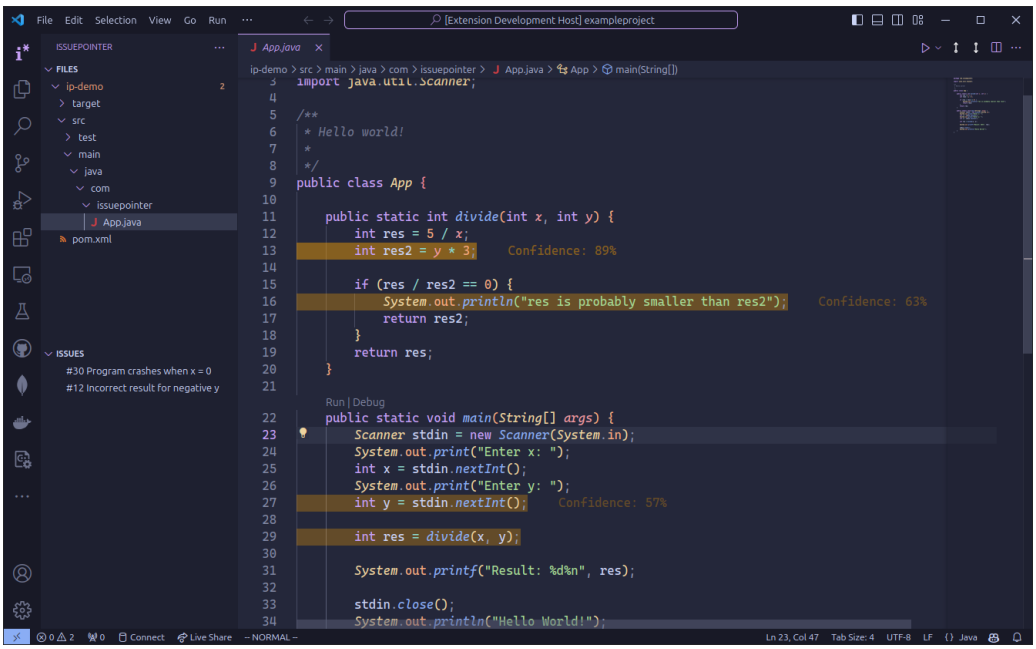

Figure 4: Mock-up of IssuePointer Visual Studio Code extension user interface

Shortcuts will be available for receiving suggestions from Bug Localization and Automated Program Repair, which will facilitate the use of the project features.

Feedback Screen As the clients submit feedback and the software keeps track of its performance, it updates its own analytics and experience site, which can provide better visualization to the clients.

1.2.2 Backend Architecture

Each solution (Duplicate Issue Detection, Bug Localization, Automated Program Repair) will be independent of each other, which will increase the performance and maintainability of the IssuePointer backend. IssuePointer will also analyze PR's that were linked to issues to generate better accuracy scores, comparing with its own solutions. All solutions will primarily be machine learning based. We're currently investigating the relevant software analytics literature for possible models/solutions to implement. Since the implementation of the backend will be influenced by the chosen models, we cannot provide a detailed description of the backend yet.

1.3 Constraints

1.3.1 Implementation Constraints

- Implementing Issue Pointer (IP) requires retrieving the list of changes made to a project over time and programmatically retrieving bug reports. This constrains the implementa-

tion to support only projects that are version-controlled with Git.

- Issue Pointer will only support projects that use GitHub Issues or Jira for bug tracking.
- The project will be implemented to allow multiple frontend modules to interact with the backend.
- The implementation of IP will contain plugins for GitHub and Jira that reply to new issues as a frontend module.
- Implementation of IP will contain an IDE extension that provides in-IDE assistance.

1.3.2 Economic Constraints

- IssuePointer will be implemented using open-source and/or free-to-use libraries and frameworks since we do not have the budget for paid frameworks and libraries.

1.4 Professional and Ethical Issues

- IP will need access the data written by the other contributors of the project to use. Therefore, we need to get permission from the contributors and provide a transparent disclosure about how we are going to use this data.
- Due to the need of AI&ML models', the general ethical discussions on artificial intelligence should also be considered for the implementation.

2 Design Requirements

2.1 Functional Requirements

2.1.1 Duplicate Reports

- Users should be able to see the other possible duplicate issues directly on the screen as they open the bug report.
- Possible duplicate issues with high confidence levels should be highlighted in green, along with their confidence ratio. Issues with moderate or low levels of confidence can be included as well with yellow or red colors -sorted by their confidence ratio- as long as the screen is able to display all issues.
- The user should be able to go to the displayed issue by clicking on its text.
- If the user confirms that our suggestion was right and the issue was a duplicate, they should be able to mark it as a duplicate. In this scenario, the system should automatically label the bug report as a duplicate and insert a reference link to the matching issue.

2.1.2 Bug Localization

- Every possible line interval should display its confidence level.
- Each suggested line interval should be navigable using next/previous icons similar to popular IDEs that allow users to navigate conflicts during merge operations.

2.1.3 Automated Program Repair

The users should be able to preview suggestions for repairing the detected bugs once they enter a keyboard shortcut.

2.2 Non-functional Requirements

2.2.1 Accessibility

Since IssuePointer's frontend is designed to be modular and integrated into existing software and websites (Github Issues, IDEs), IssuePointer should be accessible from any device capable of accessing GitHub. The full feature set should be available only for personal computers.

2.2.2 Modularity

The IssuePointer components should be modular and adaptable to different user interfaces (GitHub Issues, IDEs) or data sources (GitHub Issues, Jira)

2.2.3 Privacy

IssuePointer should not leak private code obtained from the users' repositories to unauthorized parties.

2.2.4 Security

IssuePointer should not leak authorization tokens from the users to third parties.

2.2.5 Performance

As the developers will possibly halt development while they try to find a solution, IssuePointer should be fast enough to find related issues.

3 Feasibility Discussions

3.1 Market & Competitive Analysis

CodeRabbit [1]

This tool analyzes the pull requests and then suggests code fixes. It is directly embedded into the request comment, the user is able to interact with the bot using natural language. Unlike our goal, the bot generates fixes by itself; it is unknown if CodeRabbit analyzes the pre-existing codebase of the projects.

GitHub Copilot for Pull Requests [2]

This suggestion has some live capabilities, such as auto-writing some PRs. They also suggested PR repair capability, but it is yet to be implemented.

[1] https://coderabbit.ai/
[2] https://githubnext.com/projects/copilot-for-pull-requests

Algolia [3]

Algolia seems to do what we plan to do; the only thing is that it is not open-sourced, and there is no public use of it. They probably used a general AI model that was not tailored for this task.

3.2 Academic Analysis

Contextual Approach Towards More Accurate Duplicate Bug Report Detection[4]

The paper titled 'A Contextual Approach Towards More Accurate Duplicate Bug Report Detection' (available on IEEE Xplore) argues that existing literature on duplicate bug detection primarily relies on Information Retrieval systems, such as BM25F. The authors extend this approach by incorporating 'contextual knowledge,' which entails utilizing prior information on software quality and architecture. This integration leads to an improvement in the rate of duplicate bug detection. The authors also introduce a refined definition of 'duplicate bugs,' which includes bugs that originate from the same code block but manifest under different scenarios. Additionally, they provide a comprehensive methodology detailing the implementation of the 'contextual knowledge' component in their system.

Bug Locator [5]

Bug Locator is a tool for locating relevant source files for a given bug report. It is published along with a paper. Since we want to find relevant line intervals rather than the source file itself, it is not exactly what we aim to do.

The tool finds the files by matching the texts between the bug report and the source code. It parses the bug report and source files into lexical tokens. The name of the source file, variables, and functions are used to parse a source file. Typical parsing enchantments are also applied: If the raw token is a concatenation of multiple words, it is divided into individual words, etc. So, the tool relies on developers using proper naming conventions, which is a reasonable expectation for a professional software development environment.

Improving bug localization using structured information retrieval [6]

Very similar to Bug Locator, but it performs better by using structured information. Bug Locator treats every token as the same, whether it's extracted from the class name, function, or variable it doesn't matter. In the paper, they perform token matching with different token types and they sum up later. In this way, class name tokens' contribution doesn't diminish among 100 variable tokens.

Bug Localization with Combination of Deep Learning and Information Retrieval [7]

Most promising one yet. The paper uses 2 DNNs for bug localization. First one is used for finding relevancy between the bug report and source file tokens. Second DNN is used for obtaining the final relevancy score between the bug report and the source file. It uses the result of the first DNN among with other metrics obtained such as version, last modified time etc.

[3] https://www.algolia.com/blog/engineering/building-a-github-issue-recommendation-bot-with-algolia/
[4] https://ieeexplore.ieee.org/document/6624026
[5] https://ieeexplore.ieee.org/document/6227210
[6] https://dl.acm.org/doi/10.1109/ASE.2013.6693093
[7] https://dl.acm.org/doi/pdf/10.1109/ICPC.2017.24

Practitioners' Expectations on Automated Fault Localization [8]

A paper that investigates developers' expectations from fault localization tools.
There is also another survey for APR [9]. Tables 9, 12-15, and section 6.5 might be useful.

CrossFix: Resolution of GitHub issues via similar bugs recommendation [10]

A study that has similar aims to IssuePointer, their tool extracts information from new issues in the repository (which is called the driver in the paper), which is then used to generate queries to detect similar resolved issues (issues with the same results, same libraries, same errors, or the same configuration) in other open-source projects on GitHub. The retrieved issues are then ranked internally to assess their relevance. The study's methods could be employed in IssuePointer to search only for issues local to the project at hand. The relevance ranking algorithms in CrossFix could also prove useful in ranking past issues for relevance to the issue at hand.

Duplicate Bug Report Detection Using Dual-Channel Convolutional Neural Networks [11]

This paper uses a Dual Channel CNN for duplicate/similar bug report detection. After parsing the unstructured information(title, description) into word vectors, a matrix is created for each bug report using the vectors along with structured information. Every report is paired with another bug report and their extracted association features are compared within the DC-CNN.

Structured information can be optional (0 will be given if it is not available), therefore, the model can utilize different kinds of information for different repositories.

This paper achieves a significant accuracy score of 0.95. Considering that we will be suggesting multiple bug reports, our match score will be even greater. Hence we decided that this method can be used for the project's Similar Bug Report Detection part.

[8] https://dl.acm.org/doi/10.1145/2931037.2931051
[9] https://ieeexplore.ieee.org/document/9842369
[10] https://doi.org/10.1002/smr.2554
[11] https://dl.acm.org/doi/10.1145/3387904.3389263